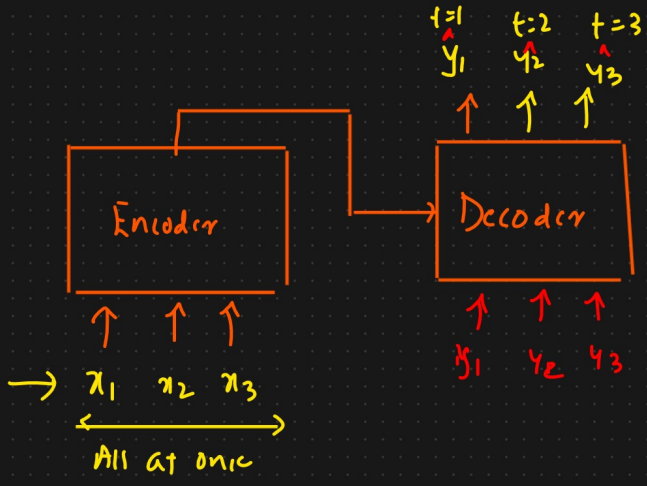


$\langle y_1, y_2, y_3 \rangle$

① Training Mechanism

② Inference Mechanism

⑧ Masked Multi Head Self Attention

① I/P Embedding And Positional Embedding ✓ → Zero padding → Sequence length Equal

② Linear Projection for Q, K, V

③ Scaled Dot Product Attention

✓④ Mask Application } ⇒ Try to understand the imp.

⑤ Multi Head Attention

⑥ Concatenation And Final Linear Projection

⑦ Residual connection And Layer Normalization

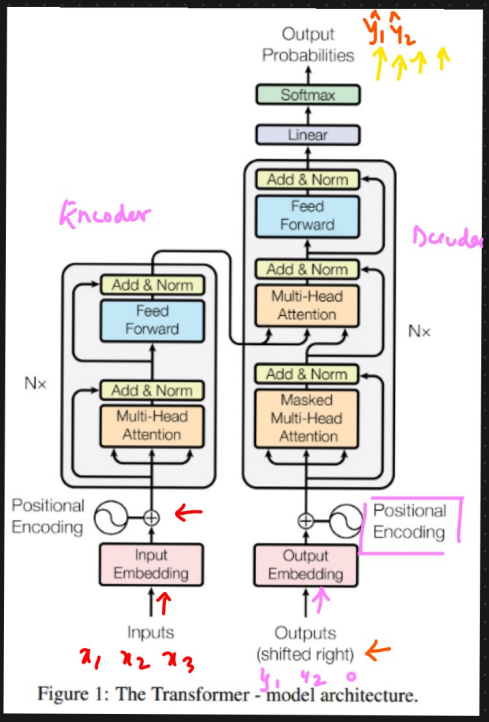


Figure 1: The Transformer - model architecture.

Dataset

Eng: $\langle x_1, x_2, x_3 \rangle$

Hindi: $\langle y_1, y_2 \rangle$ ← ⇓ $\langle y_1, y_2, 0 \rangle$ ↓ Zero padding

① Linear Projections Q, K, V

② Scaled Dot Product Attention

③ Mask Application → Look Ahead Mask, → Padding Mask

Masked Multi Head Attention.

I/P [4 5 6 7] O/P [1 2 3]

[1 2 3 0]

↓

4 dimension vector

i) Input Embedding and Positional Encoding

Output Embedding [STEP 1]

$$\begin{bmatrix} [0.1, 0.2, 0.3, 0.4], \\ [0.5, 0.6, 0.7, 0.8], \\ [0.9, 1.0, 1.1, 1.2], \\ [0.0 \quad 0.0 \quad 0.0 \quad 0.0] \end{bmatrix} + PE \Rightarrow 0$$

Step 2 : Linear Projection for Q, K, and V.

$W_Q = W_K = W_V = I$

Create query (Q), Key(K) and value(V) vectors

Q = Output Embedding * W_Q = Output Embedding

K = " * W_K = "

V = " * W_V = "

$$Q = K = V = \begin{bmatrix} [0.1, 0.2, 0.3, 0.4], \\ [0.5, 0.6, 0.7, 0.8], \\ [0.9, 1.0, 1.1, 1.2], \\ [0.0 \quad 0.0 \quad 0.0 \quad 0.0] \end{bmatrix} \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{bmatrix}^T \begin{bmatrix} 0.5 \\ 0.6 \\ 0.7 \\ 0.8 \end{bmatrix} \cdot \begin{bmatrix} 0.9 \\ 1.0 \\ 1.1 \\ 1.2 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

③ Scaled Dot Product Attention Calculation

$$\text{Scores} = Q * K^T / \sqrt{dk}$$

$$= Q * K^T / 2.$$

$$\begin{bmatrix} [0.1*0.1 + 0.2*0.2 + 0.3*0.3 + 0.4*0.4, \\ 0.1*0.5 + 0.2*0.6 + 0.3*0.7 + 0.4*0.8, \\ 0.1*0.9 + 0.2*1.0 + 0.3*1.1 + 0.4*1.2 \\ 0.1*0 + 0.2*0 + 0.3*0 + 0.4*0] \end{bmatrix}$$

Scores : [[0.3, 0.7, 1.1, 0.0]
[0.7, 1.9, 3.1, 0.0]
[1.1 3.1 5.1 0.0]
[0.0 0.0 0.0 0.0]]

[ ]
[ ]
[ ]]

⑥ Masked Application

It helps manage the structure of the sequences being processed and ensures the models behaves correctly during training And Inferencing

Reasons

① Handling Variable length Sequences with Padding MASK

Purpose

① To handle sequences of different length in batch

② To ensure that padding tokens, which are added to make sequences of uniform length, do not affect the model prediction

Eg: i/p ← Sequence 1 [1, 2, 3]

o/p ← Sequence 2 [4, 5, 0]

→ 100.

0 is the padding token

→ Influence the Attention Mechanism

⇓

lead to Incorrect or biased predictions.

o/p

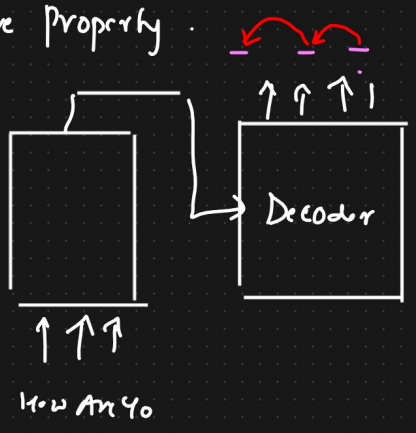
→ 4, 42 0 0 0 0 0

A padding mask ⇒ The tokens Are ignored.

Masking → Padding Mask
→ Look Ahead Mask.

Padding Mask [1 1 1]
[1 1 0]

② Look Ahead Mask → Maintain Auto Regressive Property.



① To ensure that each position in the decoder output sequence can only attend to the previous position, but no future position

② Sequence → Language Modelling, Translation

Eg: [4, 5, 0] → [1, 1, 0] → 1D Mask.

Convert 1D to 2D Mask

For each token in the sequence, the mask should indicate which tokens it can attend to.

$$\rightarrow \begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Token 1 attends to token 1, 2

Attention Mechanism ⇐ 1, 2

③ Look Ahead Mask → Decoder Output

$$\begin{bmatrix} [1 & 0 & 0] \\ [1 & 1 & 0] \\ [1 & 1 & 1] \end{bmatrix}$$

④ Combine Padding And Looking Ahead Mask

Element Wise multiplication of 2 mask

$$\text{Combine Mask} = \begin{bmatrix} [1, 0, 0] \\ [1, 1, 0] \\ [0, 0, 0] \end{bmatrix}$$

④ Mask

Scores :
$$\begin{bmatrix} [0.3, 0.7, 1.1, 0.0] \\ [0.7, 1.9, 3.1, 0.0] \\ [1.1 \quad 3.1 \quad 5.1 \quad 0.0] \\ [0.0 \quad 0.0 \quad 0.0 \quad 0.0] \end{bmatrix}$$

Look Ahead Mask

$$\begin{bmatrix} [1 & 0 & 0 & 0] \\ [1 & 1 & 0 & 0] \\ [1 & 1 & 1 & 0] \\ [1 & 1 & 1 & 1] \end{bmatrix}$$

Padding Masking [extended to 2D Format]

$$\begin{bmatrix} [1,1,1,0] \\ [1,1,1,0] \\ [1,1,1,0] \\ [0,0,0,0] \end{bmatrix}$$

Combined Mask : Look Ahead Mask * Padding Mask

$$\begin{bmatrix} [1*1, 0*1, 0*1, 0*0] \\ [1*1, 1*1, 0*1, 0*0] \\ [1*1, 1*1, 1*1, 0*0] \\ [1*1, 1*1, 1*1, 1*0] \end{bmatrix} = \begin{bmatrix} [1,0,0,0], \\ [1,1,0,0], \\ [1,1,1,0], \\ [1,1,1,0] \end{bmatrix} \Rightarrow \begin{bmatrix} [1,-\infty,-\infty,-\infty], \\ [1,1,-\infty,-\infty], \\ [1,1,1,-\infty] \\ [1,1,1,-\infty] \end{bmatrix}$$

Masked Score

$$\begin{bmatrix} [0.3, -\infty, -\infty, -\infty] \\ [0.7, 1.9, -\infty, -\infty] \\ [1.1, 3.1, 5.1, -\infty] \\ [0.0, 0.0, 0.0, -\infty] \end{bmatrix}.$$

Zero out the influence when the Softmax is applied.

Attention weights.

⑧ Softmax

Softmax Score = Softmax (Masked Scores)

$$= [[1.0, 0.0, 0.0, 0.0],$$
$$[0.3, 0.7, 0.0, 0.0],$$
$$[0.1, 0.3, 0.6, 0.0],$$
$$[1.0, 0.0, 0.0, 0.0]]$$

⑧ Weigth Sum of Value

Attention O/p = Softmax Scores * V.

Masking

Masking in the transformer architecture is essential for several reasons. It helps manage the structure of the sequences being processed and ensures the model behaves correctly during training and inference. Here are the key reasons for using masking:

1. Handling Variable-Length Sequences with Padding Mask

Purpose
To handle sequences of different lengths in a batch.
To ensure that padding tokens, which are added to make sequences of uniform length, do not affect the model's predictions.

2. Maintaining Autoregressive Property with Look-Ahead Mask

Purpose
To ensure that each position in the decoder output sequence can only attend to previous positions and itself, but not future positions.
This is crucial for sequence generation tasks like language modeling and translation, where the model should not have access to future tokens when predicting the current token.

④ Encoder Decoder Multi Head Attention

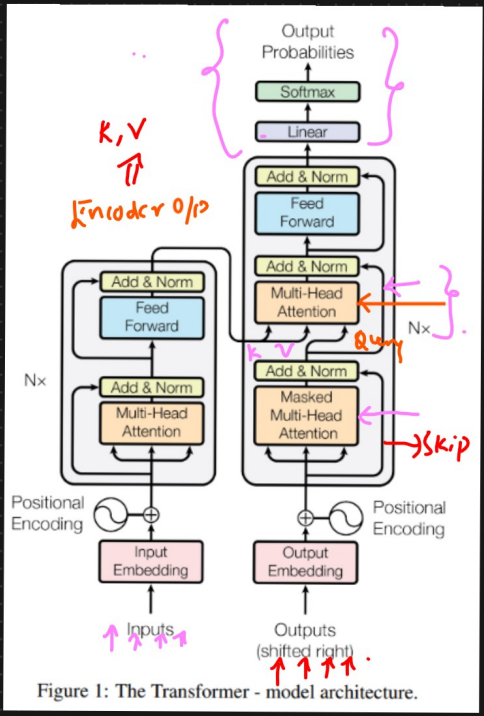


Figure 1: The Transformer - model architecture.

① Encoder O/p → Set of Attention vector K & V

② Masked Multihead → Attention vector Q {Query Vector}

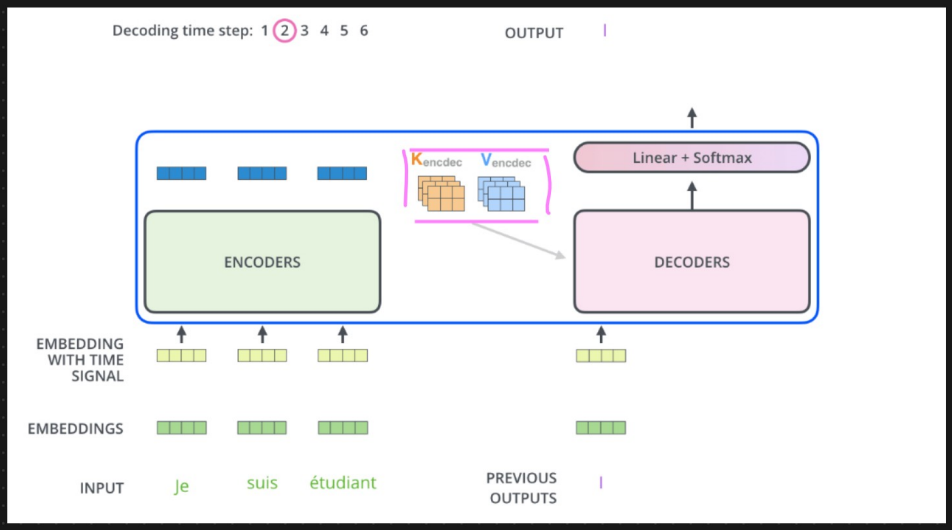
These are to be used by each decoder in its "encoder – decoder" attention layer

⇓

Helps the Decoder to focus on appropriate places in the i/p sequence

Dataset

i/p $\langle x_1\ x_2\ x_3 \rangle$ O/p $\langle y_1\ y_2\ y_3 \rangle$.



④ The Final Linear And Softmax Layer

{Vectors ⟶ o/p word}

{BLOG} ⇒ Transformers

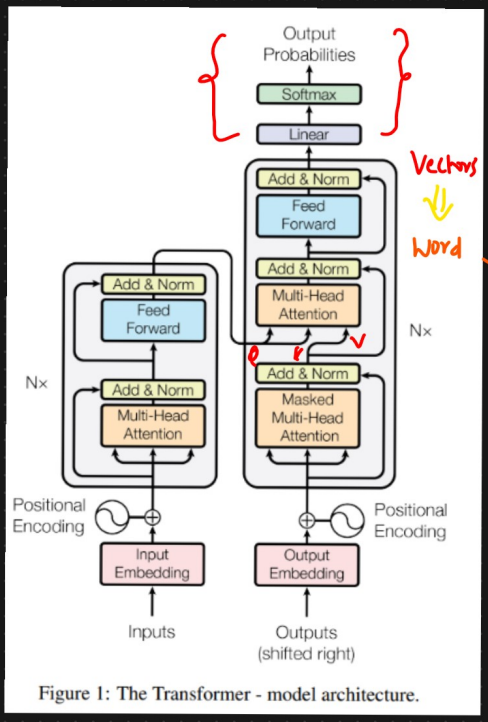
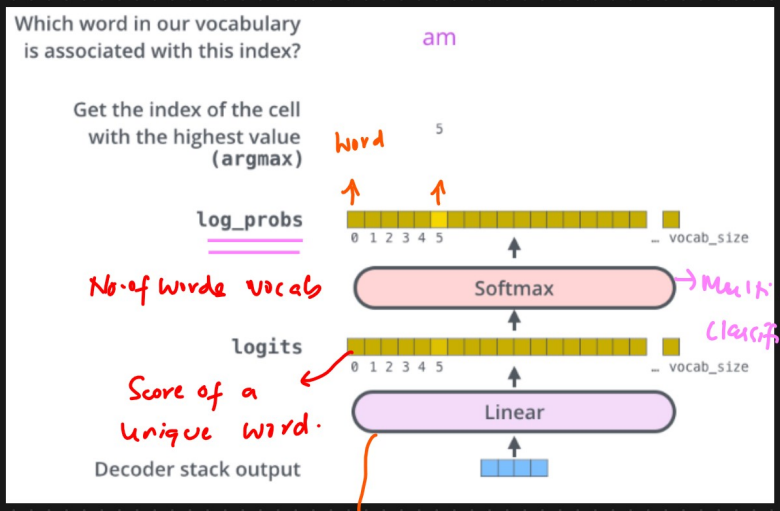


Figure 1: The Transformer - model architecture.



Linear ⇒ The linear layer is a simple fully connected neural n/w that projects the vector produced by the stack of Decoder ⇒ logits vector

⇒

Model ⇒ 10,000 ⇒ vocabulary ⇒ logits vector = 10000 cells wide

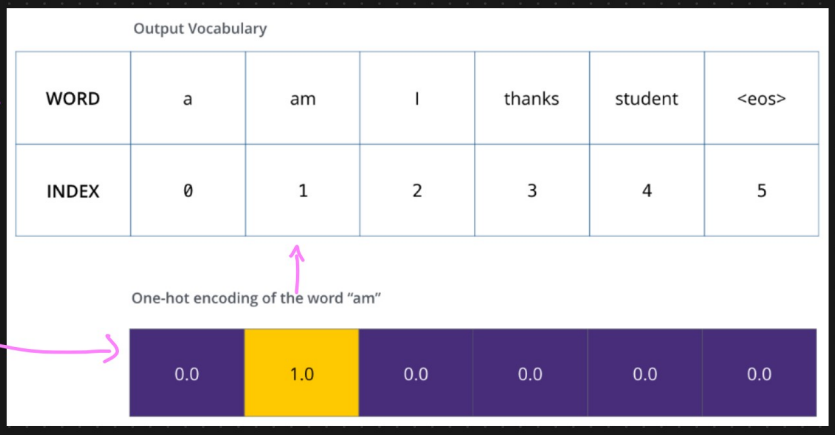
② Softmax layer turns those scores into probabilities (all add upto 1.0).

The cell with the highest probability is chosen, and the word associated with it is produced as the o/p. ⇒ time stamp.

Recap of Training

Output Vocabulary

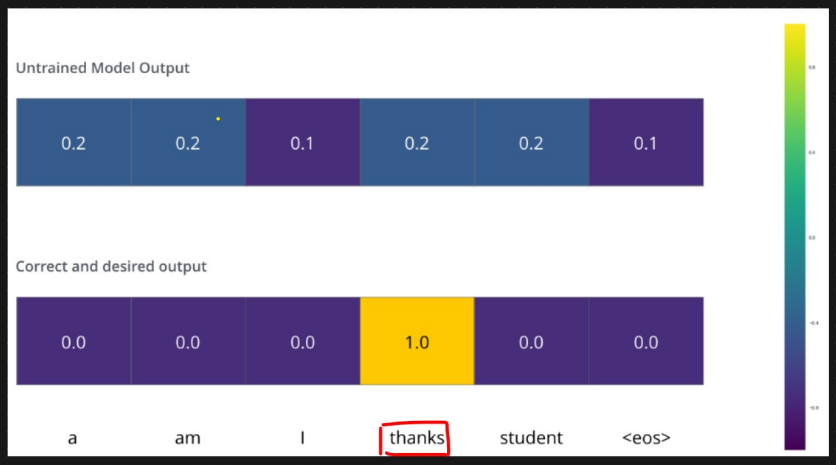
WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5



OHE ←

Merci → Thanks

I am a student <eos>.



⇒ Loss function ↓↓.

Back Propogation

i/p

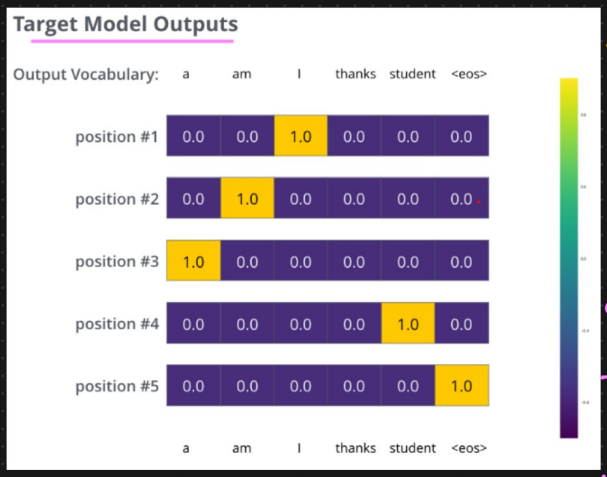
a, am, i, student

o/p

I am a student

↓ ↓ ↓ ↓

OHE OHE OHE OHE



Loss